Reviewer Pack — Symmetric Polynomial Invariant Degree Bounds AC⁰ PARITY Dept...

Entry 0581f3c74a91 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 08:38:11 UTC

Symmetric Polynomial Invariant Degree Bounds AC⁰ PARITY Depth

Entry ID: `0581f3c74a91` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 08:38:11 UTC

1. Conjecture

Field A (mathematical branch): Invariant Theory **Field B** (complexity object): AC⁰ Circuit Size for PARITY

Statement:

> For any AC⁰ circuit C computing PARITY on n inputs, the minimal degree of a symmetric polynomial invariant under the action of the symmetric group S_n on the input variables is Ω(log n).

Rationale (proposer's reasoning):

> Symmetric invariants capture structural symmetries of PARITY functions. By linking their degree to circuit depth, we may extract combinatorial constraints bypassing random restriction arguments. The symmetric group's action reflects PARITY's inherent symmetry, making invariants a natural lens for lower bounds.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8ce87d5adc399703`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def symmetric_polynomial_degree(n):
        if n <= 1:
            return 0
        return 1 + symmetric_polynomial_degree(n - 2)

    degree = symmetric_polynomial_degree(40)
    metric_value = degree

    conjecture_holds = degree >= math.log(40, 2)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "symmetric_polynomial_degree",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
sted': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'symmetric_polynomial_degree', 'metric_value': 20, 'instances_tested': 1, 'co
RESULT: SUPPORTED mean=20.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge | original judge: All trials (n=1) show metric value 20, meeting the 80% support threshold with no counterexamples found. | next: Test larger n values to verify scalability of the degree bound

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	44777
2	preregistration	ollama_remote	qwen3:8b	0	0	32019
3	novelty	ollama_remote	qwen3:8b	0	0	27763
4	novelty	ollama_remote	qwen3:8b	0	0	19984
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13026
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11794
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33463
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9065

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	critic	ollama_remote	qwen3:8b	0	0	129110
10	judge	ollama_remote	qwen3:8b	0	0	20745

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 341745 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0581f3c74a91.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0581f3c74a91.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0581f3c74a91.tar.gz` (if generated)